

OS_2018-06-19

init(B1, 1)

init(B2, 1)

init(SB1_w, 1)

1) Synchronisation (30)

German version

Das Leser/Schreiber-Problem ist wie folgt definiert: Ein Datenbereich wird von mehreren Prozessen geteilt. Der Datenbereich könnte eine Datei, ein Speicherblock im Hauptspeicher oder eine Reihe von Prozessorregister sein. Eine Teilmenge der Prozesse greift nur lesend auf den Datenbereich zu (Leser) und eine andere Teilmenge der Prozesse greift nur schreibend auf den Datenbereich zu (Schreiber). Folgende Bedingungen müssen bei jedem Datenzugriff gewährleistet sein:

1. Eine beliebige Anzahl an Prozessen kann gleichzeitig vom Datenbereich lesen
2. Zu jeder Zeit darf maximal ein Prozess auf den Datenbereich schreibend zugreifen
3. Während ein Schreiber auf den Datenbereich zugreift darf kein Prozess vom Datenbereich lesen
4. Schreiber dürfen nicht verhungern: Kein Leser darf auf den Datenbereich zugreifen sobald zumindest ein Schreiber bekanntgegeben hat, dass dieser auf den Datenbereich schreiben möchte.

Implementieren Sie eine Lösung die, welche die im Initialisierungscode gegebene Semaphoren und Variablen verwendet. Zum Lesen und Schreiben auf den Datenbereich sollen die Funktionen *read()* bzw. *write()* verwendet werden.

Code for Initialization

```
semaphore    x = 1;
semaphore    y = 1;
semaphore    z = 1;
semaphore wsem = 1; //Writing semaphors
semaphore rsem = 1; //Reading semaphors
int          rc = 0;
int          wc = 0;
```

Code for Process Reader

```
void reader (void){
    while (true) {
        // Complete the code

        wait(x);
        wait(rsem);
        rc++;
        if (rc == 1) wait(wsem);
        signal(rsem);
        signal(x);

        read();

        wait(x);
        rc--;
        if (rc == 0) signal(wsem);
        signal(x);

    }
}
```

Code for Process Writer

```
void writer (void){
    while (true) {
        // Complete the code

        wait(y);
        wc++;
        if (wc == 1) wait(x);
        signal(y);

        wait(wsem);
        write();
        wait(wsem);

        wait(y);
        wc--;
        if (wc == 0) signal(x);
        signal(y);

    }
}
```

2) Uniprocessor Scheduling (20)

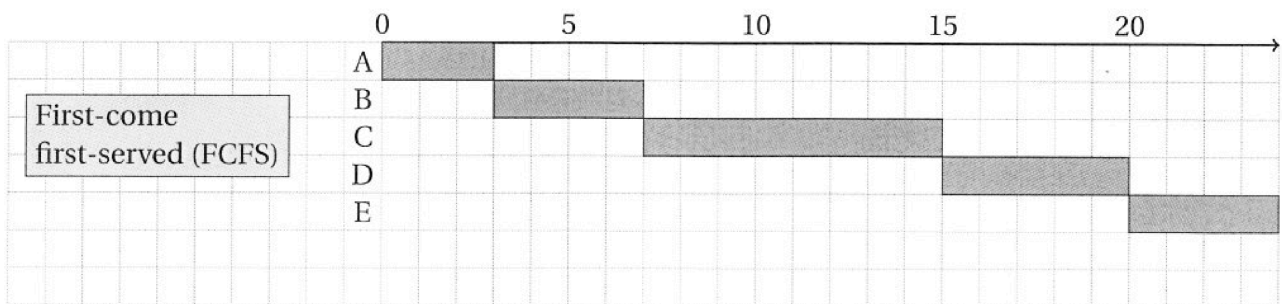
German version

Gegeben sei folgendes Taskset:

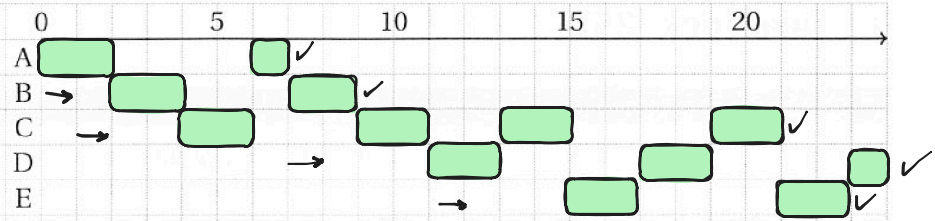
Process	Arrival Time	Service Time
A	0	3
B	1	4
C	2	8
D	8	5
E	12	4

Schedulen Sie die Tasks im untenstehenden Raster (wie am Beispiel FCFS gezeigt) für die folgenden Scheduling Algorithmen:

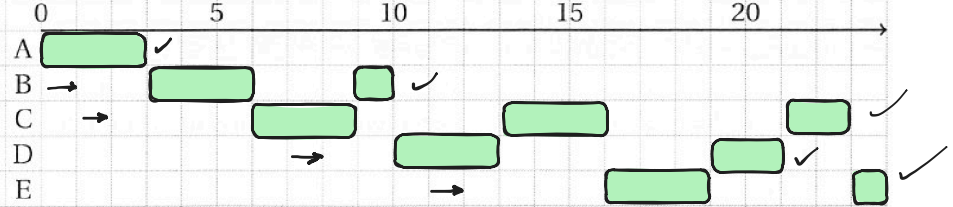
- Round robin (Zeitscheibenlänge $q = 2$ und $q = 3$)
 - Shortest process next (SPN)
 - Shortest remaining time (STR)
 - Highest response ratio next (HRRN)
- In SRT: Behandeln Sie Tasks in alphabetischer Reihenfolge, wenn Tasks dieselbe verbleibende Service-Zeit haben.
 - In RR (zum Beispiel $q=1$): Wählen Sie den neuen Task, wenn ein neuer Task ankommt und die Zeitscheibe für den vorherigen Task endet.



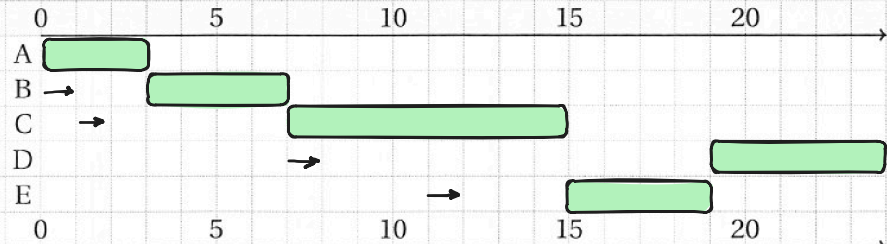
Round-robin
(RR), $q=2$



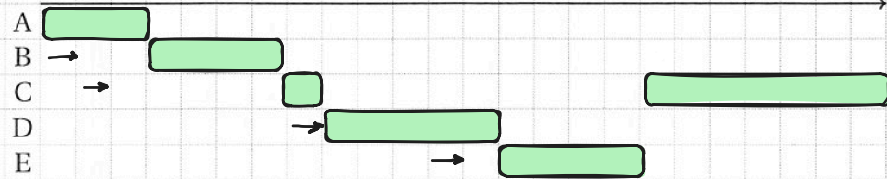
Round-robin
(RR), $q=3$



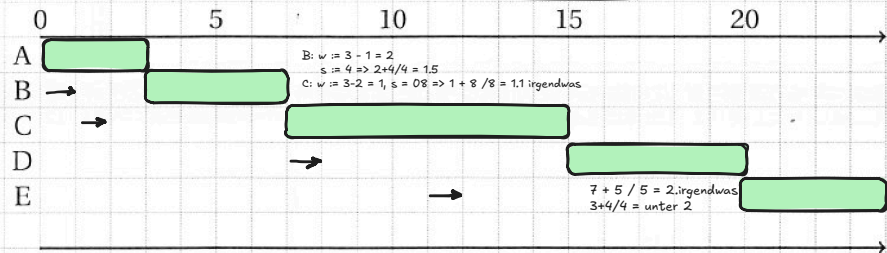
Shortest process
next (SPN)



Shortest remaining
time (STR)



Highest response
ratio next (HRRN)



3) Deadlock (25)

German version

In den folgenden Codestücken konkurrieren 3 Prozesse P0 bis P2 um 6 Ressourcen A bis F. Zeigen Sie die Möglichkeit eines Deadlocks in dieser Implementierung mittels "Resource Allocation Graph" (siehe "Example").

Process P0	Process P1	Process P2
<pre> void P0 () { // while (true) { // get(A); get(E); get(B); criticalS1(); release(B); release(E); release(A); } // } </pre>	<pre> void P1 () { // while (true) { get(C); get(D); get(B); get(A); criticalS2(); release(A); release(B); release(D); release(C); } } </pre>	<pre> void P2 () { while (true) { get (D); get (B); get (C); get (E); criticalS3(); release(E); release(C); release(B); release(D); } } </pre>

Example	Resource Allocation Graph

German version

Ändern Sie die Reihenfolge der Ressourcen-Anforderungen *innerhalb eines Prozesses* um jede Möglichkeit eines Deadlocks zu verhindern.

Process P0	Process P1	Process P2
<pre> void P0 () { // while (true) { // get(A); get(B); get(E); criticalS1(); release(E); release(B); release(A); } // } </pre>	<pre> void P1 () { // while (true) { get(A); get(B); get(C); get(D); criticalS2(); release(D); release(C); release(B); release(A); } } </pre>	<pre> void P2 () { while (true) { get (B); get (C); get (D); get (E); criticalS3(); release(E); release(D); release(C); release(B); } } </pre>

German version

Gegeben sei folgender Zustand für den "Banker's Algorithm". Snapshot zum Zeitpunkt T0:
Verfügbare Ressourcen

V:

A	B	C	D
6	3	1	9

A C N

Process	Aktuell alloziert				Max. Nachfrage				Need matrix			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	9	0	4	0	9	5	5	5	0	5	1	5
P1	0	0	0	0	2	2	3	3	2	2	3	3
P2	0	1	0	0	7	5	4	4	7	4	4	4
P3	1	0	2	0	3	3	3	2	2	3	1	2
P4	0	2	1	0	5	2	2	1	5	0	1	1
P5	0	0	1	0	3	2	2	4	3	2	1	4

1. Berechnen Sie die *Need matrix*. Ist der aktuelle Zustand sicher?

- ☒ Ja. Zeigen Sie, dass der aktuelle Zustand sicher ist indem Sie eine Deadlock freie Reihenfolge zum Ausführen der Prozesse finden. Berechnen Sie auch die Änderungen der verfügbaren Ressourcen wenn ein Prozess terminiert.

Terminierungsreihenfolge
der Prozesse

Verfügbare
Ressourcen

Prozess	A	B	C	D
P ₃	7	3	3	9
P ₁	7	3	3	9
P ₄	7	5	4	9
P ₀	16	5	8	9
P ₂	16	6	8	9
P ₅	16	6	9	9

☐ Nein. Wieso?

2. Gegeben sei die Anforderung (1,3,1,2) des Prozesses P3. Sollte dieser Anforderung stattgegeben werden?

- ☒ Ja. (Begründen anhand der Tabelle.)
☐ Nein. (Begründen anhand der Tabelle.)

Verfügbare Ressourcen

$$13 \ 12 \ 5 \ 2 \ 3 \ 12 \checkmark$$

$$13 \ 12 \ 5 \ 6 \ 2 \ 19 \checkmark$$

$$V - (1,3,1,2) =$$

A	B	C	D
5	0	0	7

Process	Aktuell alloziert				Max. Nachfrage				Need matrix			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	9	0	4	0	9	5	5	5	0	5	1	5
P1	0	0	0	0	2	2	3	3	2	2	3	3
P2	0	1	0	0	7	5	4	4	7	4	4	4
P3	2	3	3	2	3	3	3	2	1	0	0	0
P4	0	2	1	0	5	2	2	1	5	0	1	1
P5	0	0	1	0	3	2	2	4	3	2	1	4

Das ist einfach die
Tabelle von oben aber die eine
Zeile geändert

Terminierungsreihenfolge
der Prozesse

Verfügbare
Ressourcen

Process	A	B	C	D
P ₃	7	3	3	9
P ₅	7	3	4	9
P ₄	7	5	5	9
P ₂	7	6	5	9
P ₀	16	6	9	9
P ₁	16	6	9	9

wieder wie bei (1.)

4) Page Replacement (25)

German version

Ein Prozess referenziert 5 Pages, A, B, C, D, E, in folgender Reihenfolge:

D C D C E E E E E B C B A E B D

Annahme: fixed frame allocation (fixed resident set size) und einen leeren Arbeitsspeicher der Größe von 4 Frames. Benutzen Sie die folgenden 4 Tabellen, um das Verhalten der Page Replacement Policies OPT, LRU und Clock, zu vergleichen. Bitte markieren Sie in der letzten Zeile der Tabellen auftretende Page Faults *nachdem die Frames des Arbeitsspeichers initial gefüllt sind*.

OPT-Policy

	D	C	D	C	E	E	E	E	E	B	C	B	A	E	B	D
0	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D	D
1		C	C	C	C	C	C	C	C	C	C	C	A	A	A	A
2					E	E	E	E	E	E	E	E	E	E	E	E
3										B	B	B	B	B	B	B
PF	X	X			X					X			X			

LRU-Policy

	D	C	D	C	E	E	E	E	E	B	C	B	A	E	B	D
0	D	D	D	D	D	D	D	D	D	D	D	D	A	A	A	A
1		C	C	C	C	C	C	C	C	C	C	C	C	C	C	D
2					E	E	E	E	E	E	E	E	E	E	E	E
3										B	B	B	B	B	B	B
PF	X	X			X					X			X			X

Clock-Policy

	D	C	D	C	E	E	E	E	E	B	C	B	A	E	B	D
0	D	D	D	D	D	D	D	D	D	D	D	D ₁	A ₁	A ₁	A ₁	A
1		C	C	C	C	C	C	C	C	C	C	C ₁	C ₀	C ₀	C ₀	D
2					E	E	E	E	E	E	E	E ₁	E ₀	E ₀	E ₀	E
3										B	B	B ₁	B ₀	B ₀	B ₀	B
PF	X	X			X					X			X			X